

A photograph of a turtle crossing a gravel road. The turtle is positioned on the left side of the frame, facing right. Its shell is brown with yellowish-orange patterns. The road is made of light-colored gravel and stretches towards the horizon. In the background, there are green bushes, a yellow road sign, and a range of mountains under a cloudy sky. The text "Beyond Technical Debt" is overlaid in white, sans-serif font across the middle of the image.

Beyond Technical Debt

TECHNICAL DEBT

“Stuff that isn’t supposed to be there and is in the way of the stuff that is supposed to be there.”

— Building Evolutionary Architectures (p. 110), by Neal Ford, Rebecca Parsons, and Patrick Kia

LEHMAN

.....

Lehman's "Laws" of Software Evolution

Continuing Change

"a system must be continually adapted or it becomes progressively less satisfactory"

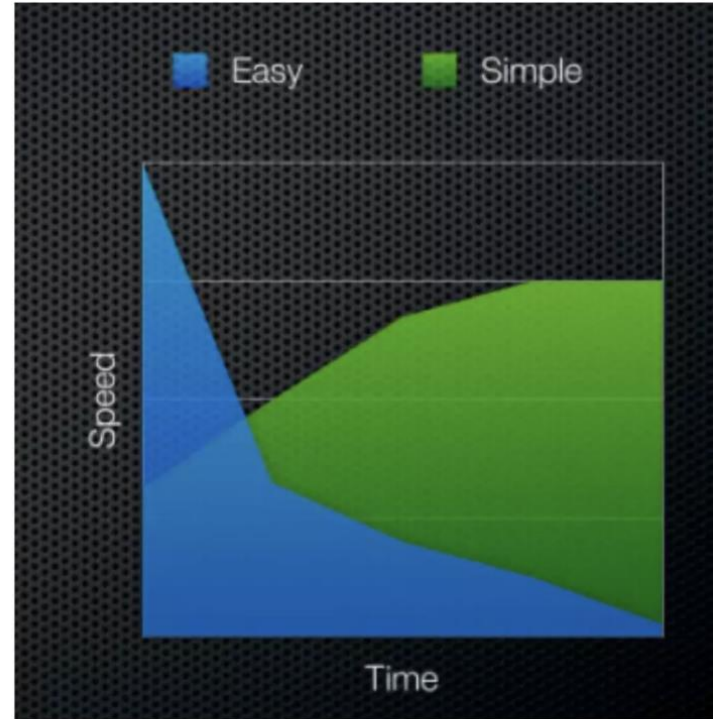
Increasing Complexity

"as a system evolves, its complexity increases unless work is done to maintain or reduce it"

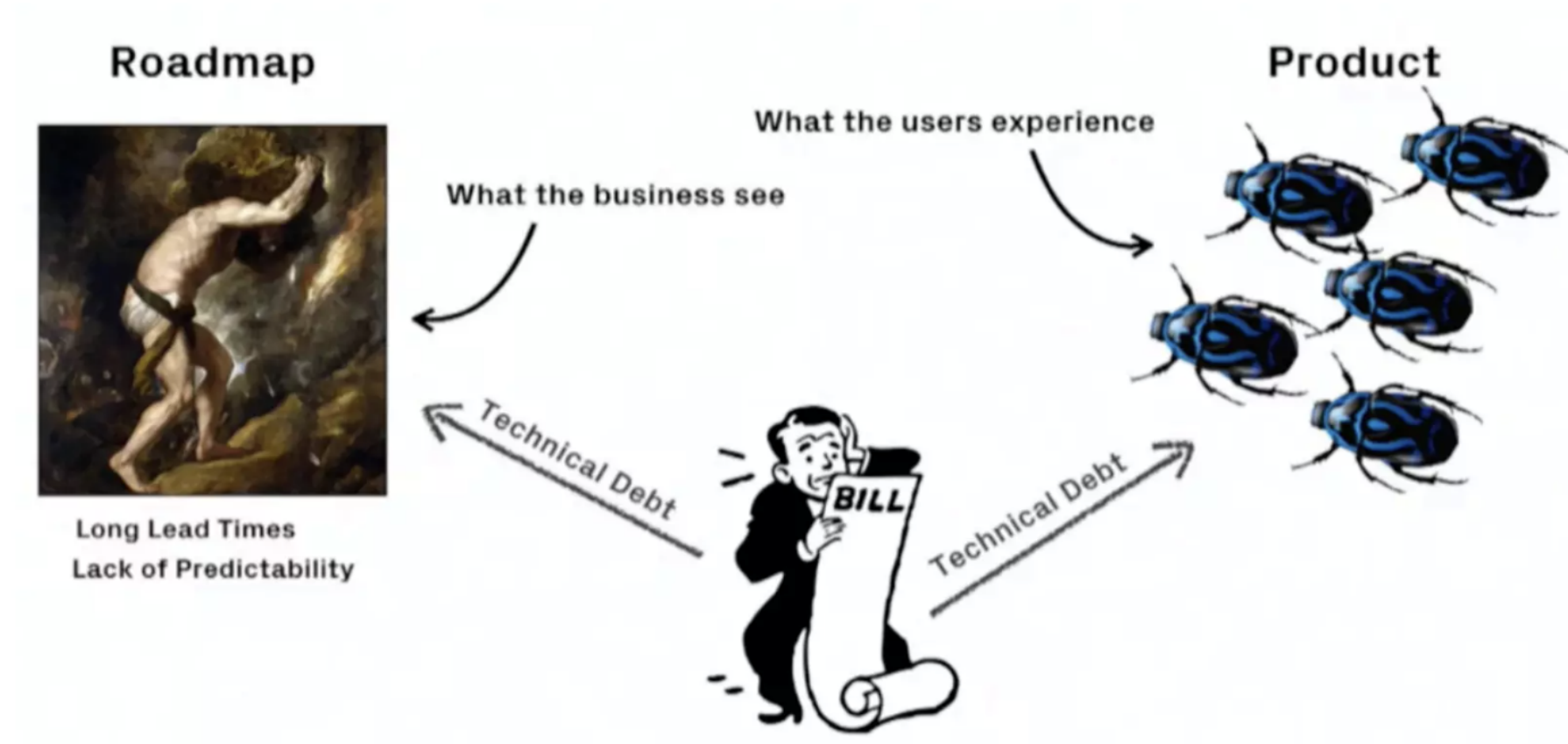
COMPLEXITY KILLS DEVELOPMENT SPEED

*“It's my contention, based on experience, that **if you ignore complexity, you will slow down.** You will invariably slow down over the long haul
...**the complexity will eventually kill you.** It will kill you in a way that will make every sprint accomplish less.*

— Rich Hickey, Simple Made Easy



TECHNICAL DEBT HAS IMPACT ON BUSINESS & PRODUCT

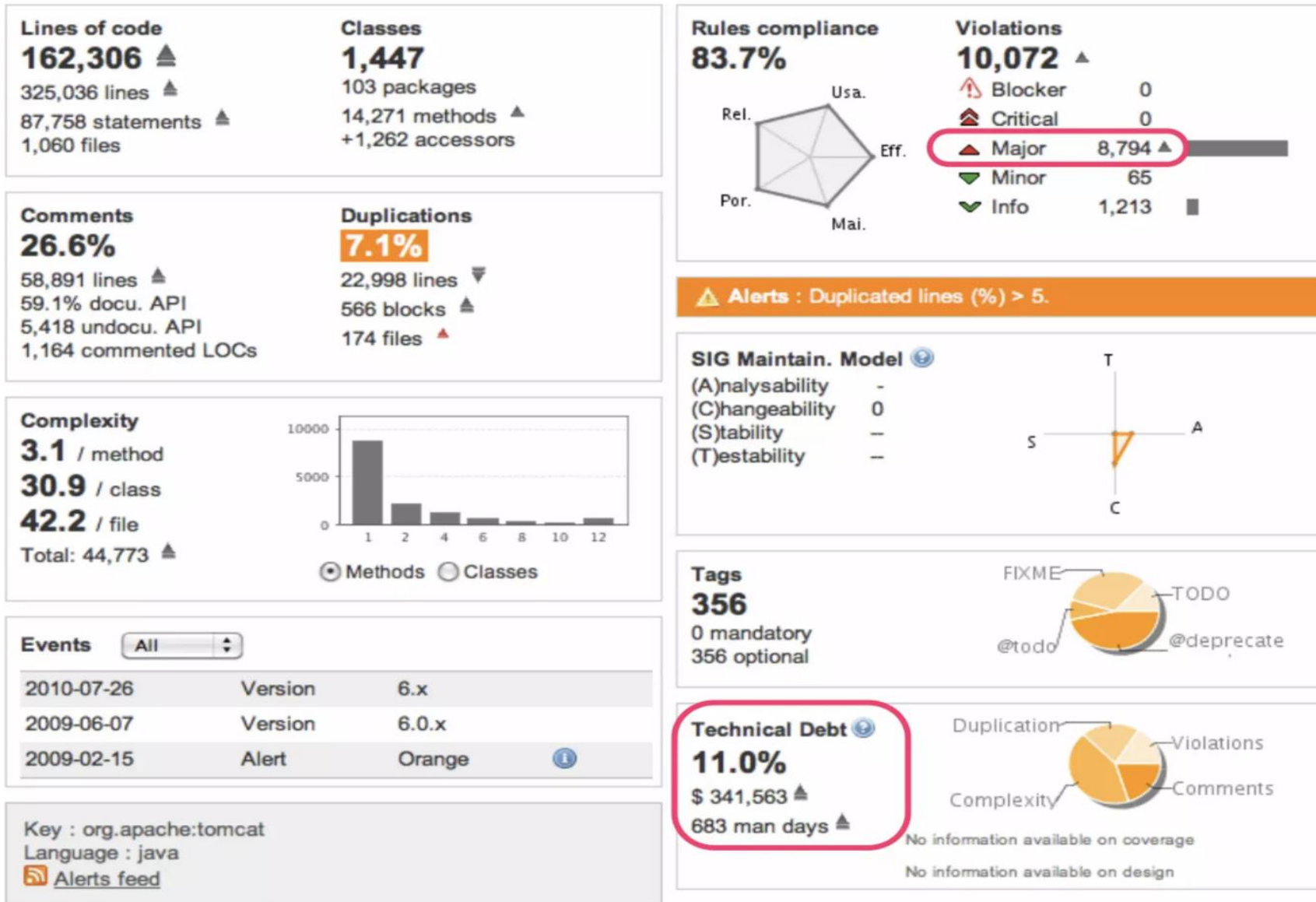




Conventional Tools

What is missing?

ACTIONABLE?



source: <http://www.austinjug.org/presentations/HeintzTechnicalDebtSonar.pdf>

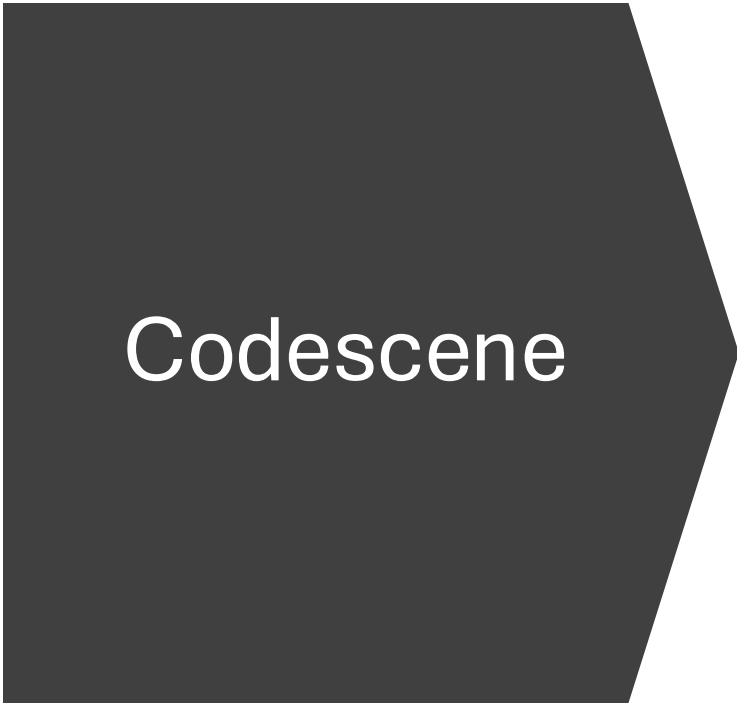
THOUSANDS OF YEARS OF TECHNICAL DEBT

- Where do you start when you want to pay it back?



Codescene

It's a "movie" rather than a "snapshot"...



Codescene

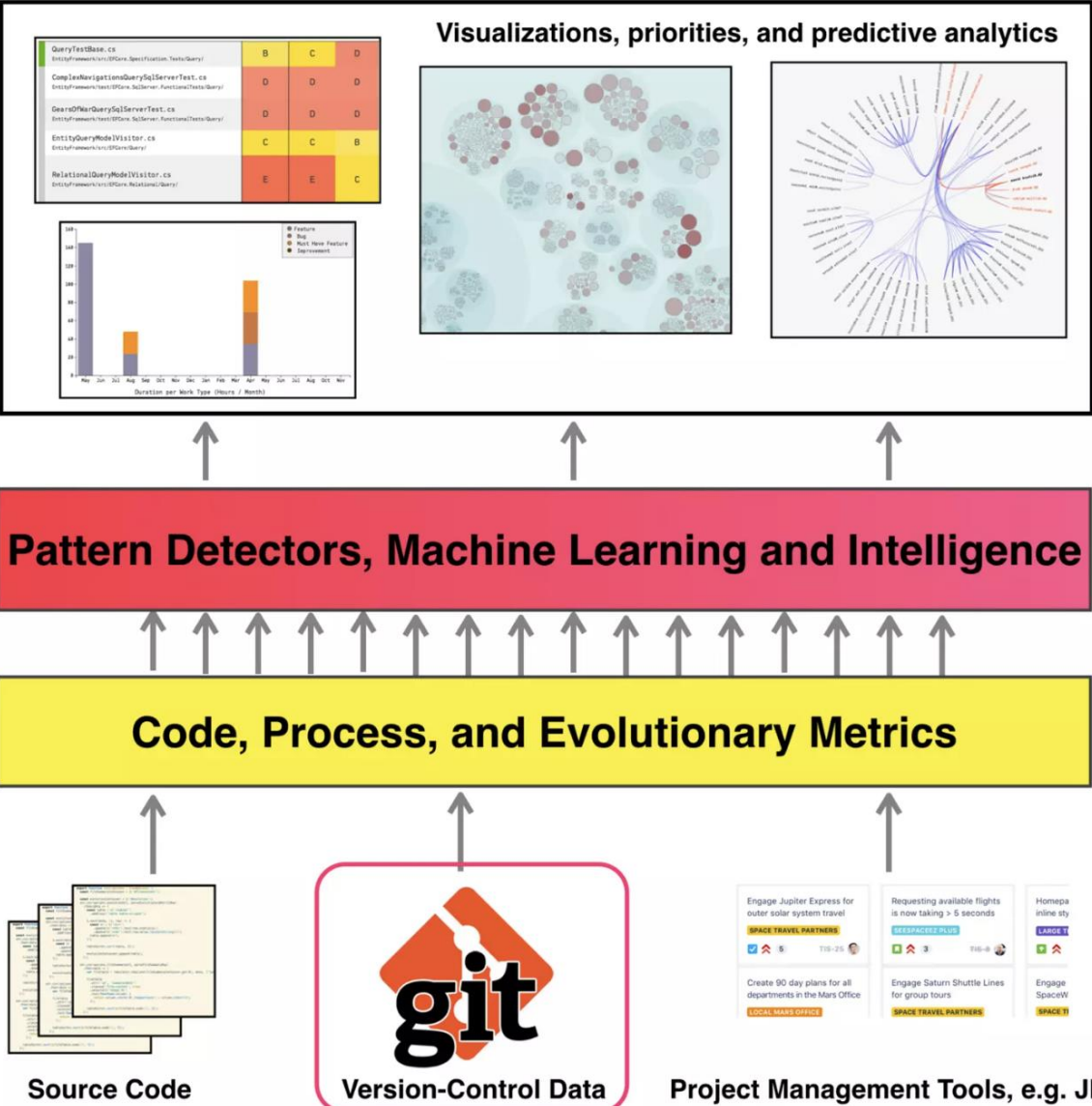
... static analysis will never be able to tell you if that excess code complexity actually matters – just because a piece of code is complex doesn't mean it's a problem.

Codescene identifies and prioritizes technical debt based on how the organization works with the code

— How Codescene Differs From Traditional Code Analysis Tools

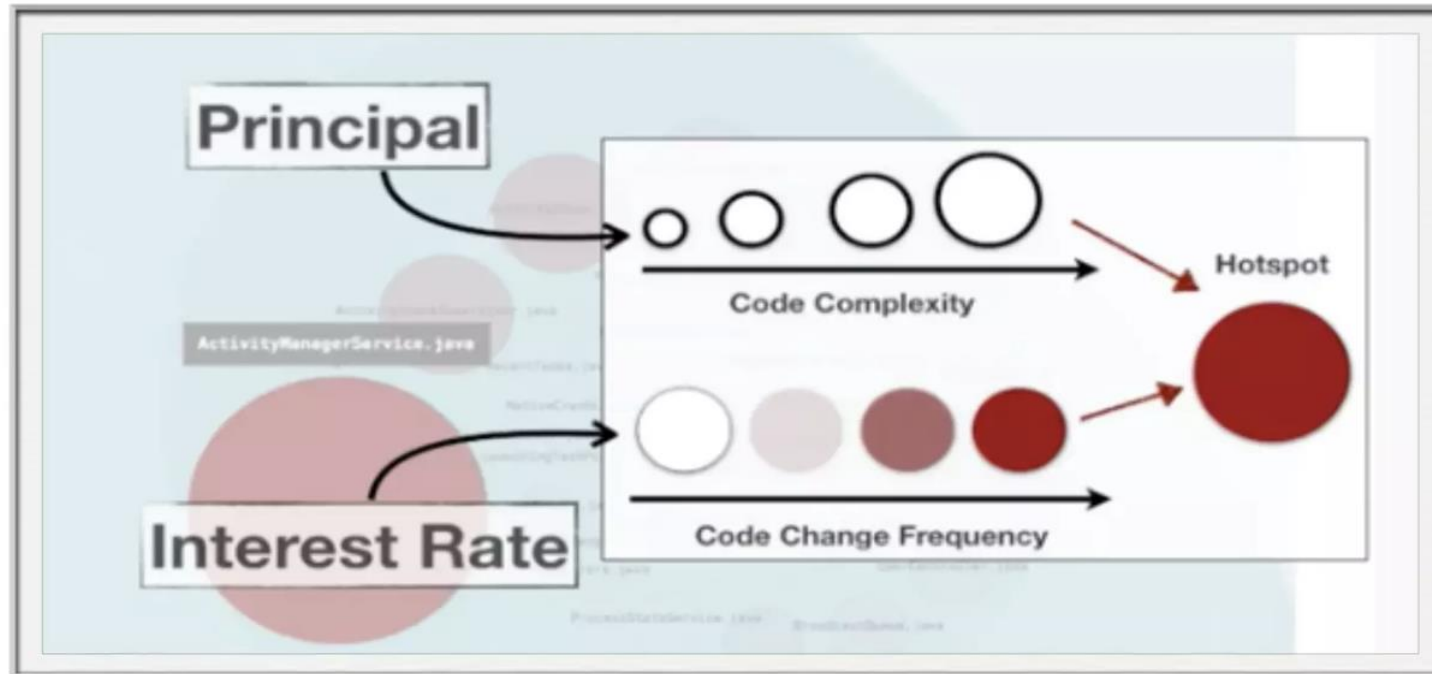
+ *Time aspect*

+ *Organization & people*



Hotspot

A hotspot is a complicated code that you have to work with often.



ReactJS

The image displays a bubble chart representing the ReactJS ecosystem. The chart is set against a light blue background with a large, faint, light green circle. The bubbles are arranged in a circular pattern, with the largest bubble, 'react-dom', at the top right. Other prominent bubbles include 'react' at the bottom left, 'react-reconciler' at the bottom right, and 'react-devtools-shared' at the bottom center. Smaller bubbles are scattered throughout, including 'react-interactions', 'react-debug-tools', 'react-refresh', 'react-is', 'react-stream', 'react-native-renderer', 'shared', 'legacy-events', 'scheduler', 'eslint-plugin-react-hooks', and 'react-devtools-shared'. The bubbles are colored in shades of blue, green, and red, and their size represents their relative importance or usage within the ecosystem.



HOTSPOTS ARE ALSO COMMON SOURCES OF BUGS

Predicting Fault Incidence Using Software Change History

Article (PDF Available) in [IEEE Transactions on Software Engineering](#) 26(7):653 - 661 · August 2000 with 420 Reads ⓘ

DOI: 10.1109/32.859533 · Source: [IEEE Xplore](#)

[↓ Cite this publication](#)



Todd L. Graves



Alan F. Karr

Jul 27.09 · RTI International



J.S. Marron

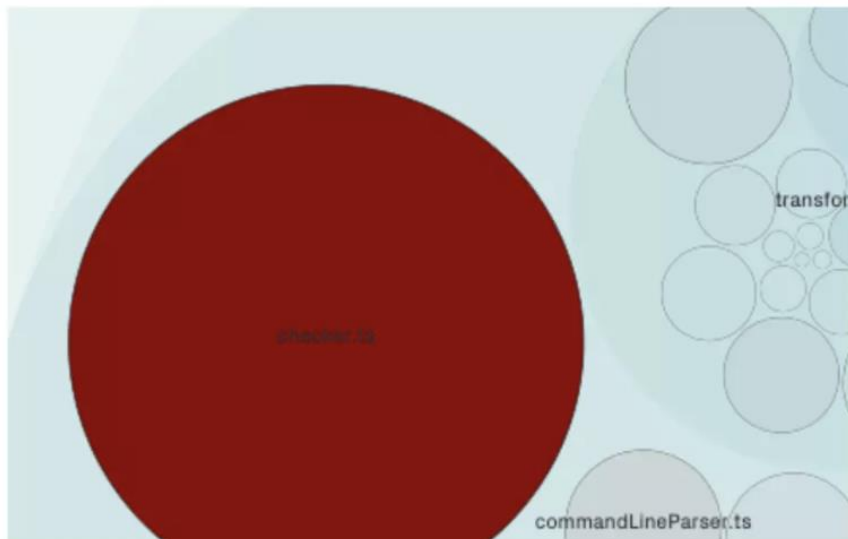


Harvey Siy

Jul 19.18 · University of Nebraska at Omaha

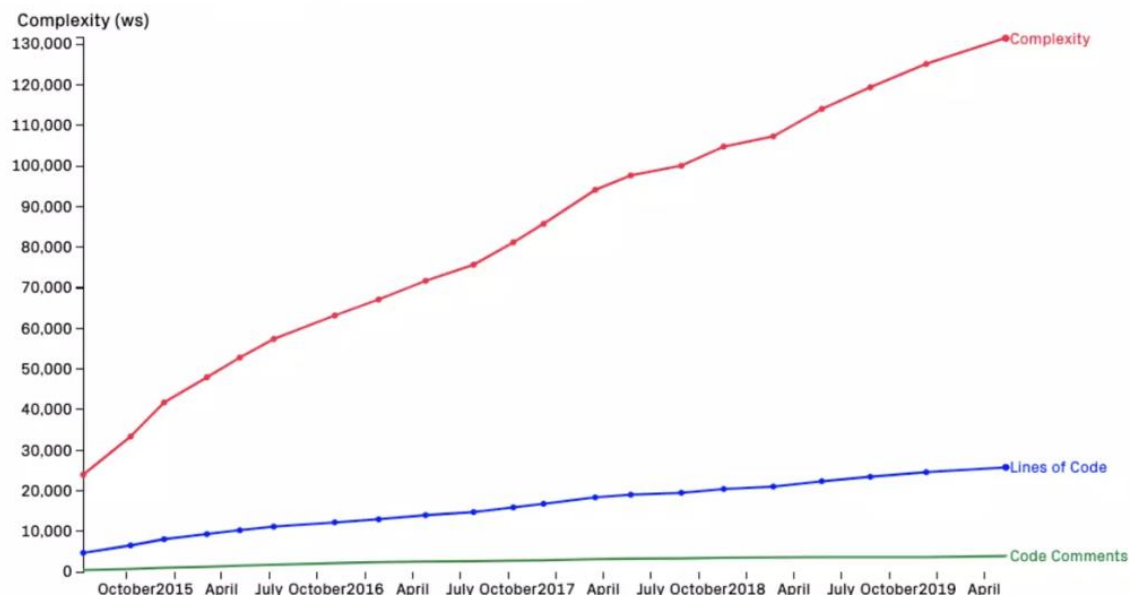
- In general, process measures based on the change history are more useful in predicting fault rates than product metrics of the code: For instance, the **number of times code has been changed is a better indication of how many faults it will contain** than is its length.
- We also compare the fault rates of code of various ages, finding that if a module is, on the average, a year older than an otherwise similar module, **the older module will have roughly a third fewer faults**. Our most successful model measures the fault potential of a module as the sum of contributions from all of the times the module has been changed, with large, recent changes receiving the most weight

WORRISOME TRENDS



Complexity Trend

[Click on a point to diff the code changes.](#)



TypeScript / TypeScript / src
/ compiler / checker.ts

Size 25774 Lines of Code

Code Health

1

Change Frequency 785 Commits

Main Author Anders Hejlsberg (26 %)

Knowledge Loss 0 %
Abandoned Code

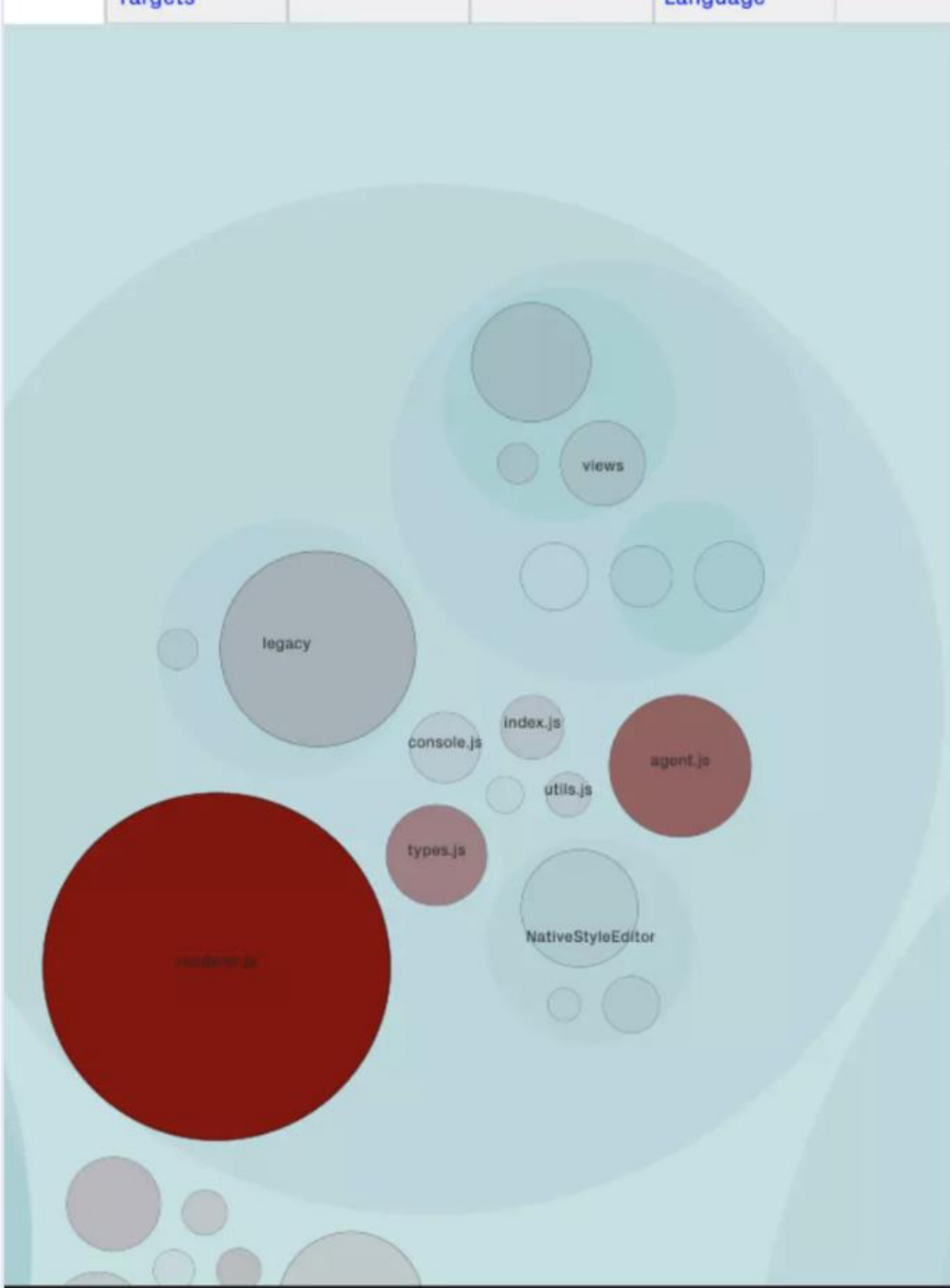
Defects 1051 (133 % Bug Fixes)

Last 0 months ago

X-Ray

Deep Dive





[react](#) / [react](#) / [packages](#)
/ [react-devtools-shared](#) / [src](#)
/ [backend](#) / [renderer.js](#)

Size	2444 Lines of Code	Code Health 5
Change Frequency	167 Commits	
Main Author	Brian Vaughn (85 %)	
Knowledge Loss	0 % Abandoned Code	
Defects	34 (20 % Bug Fixes)	
Last Modified	0 months ago	

Actions

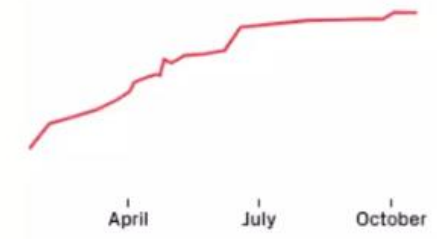
[View Code](#)

[X-Ray](#)

[Trends](#)

[Code Review](#) ▼

Complexity Trend

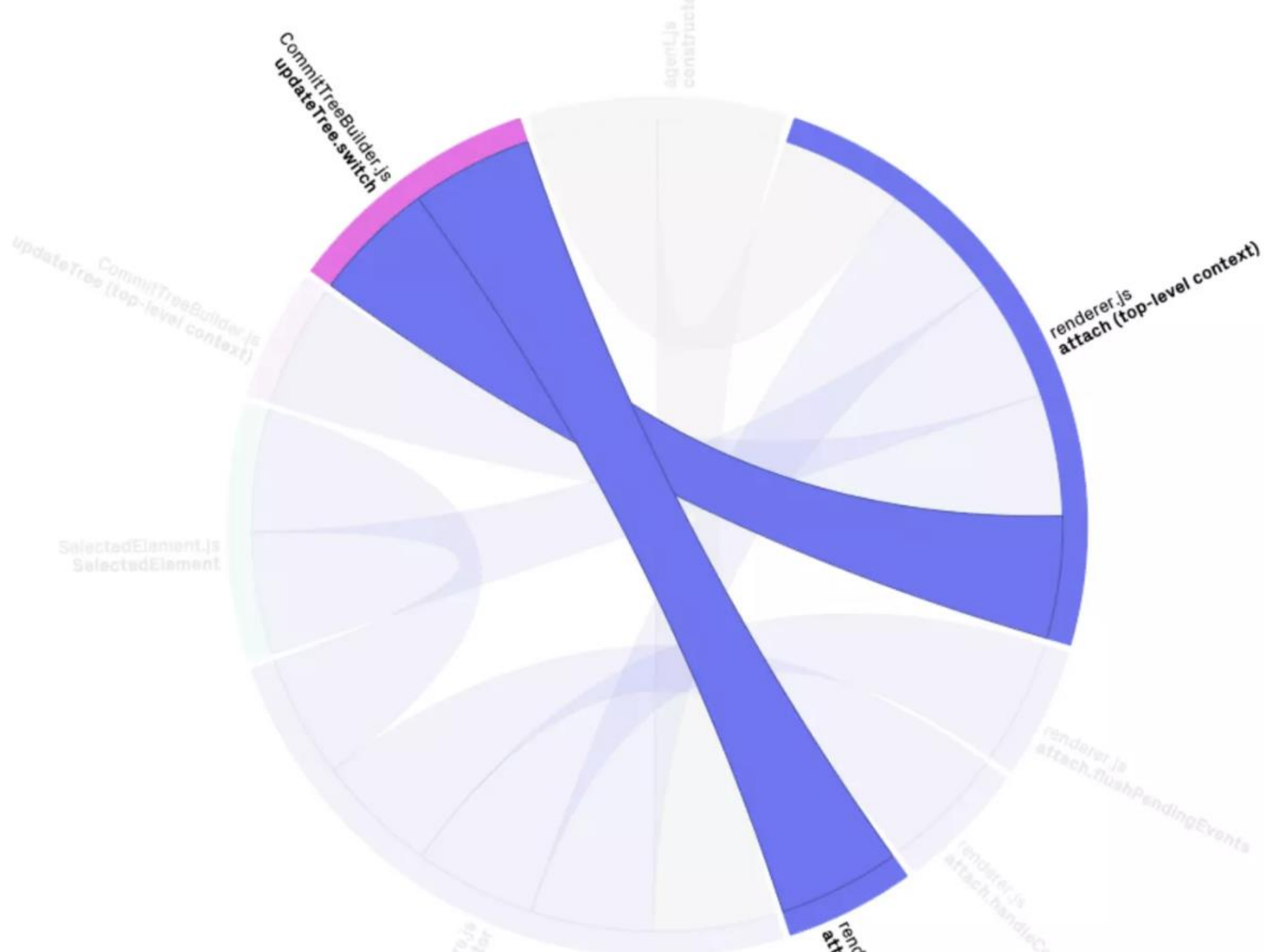


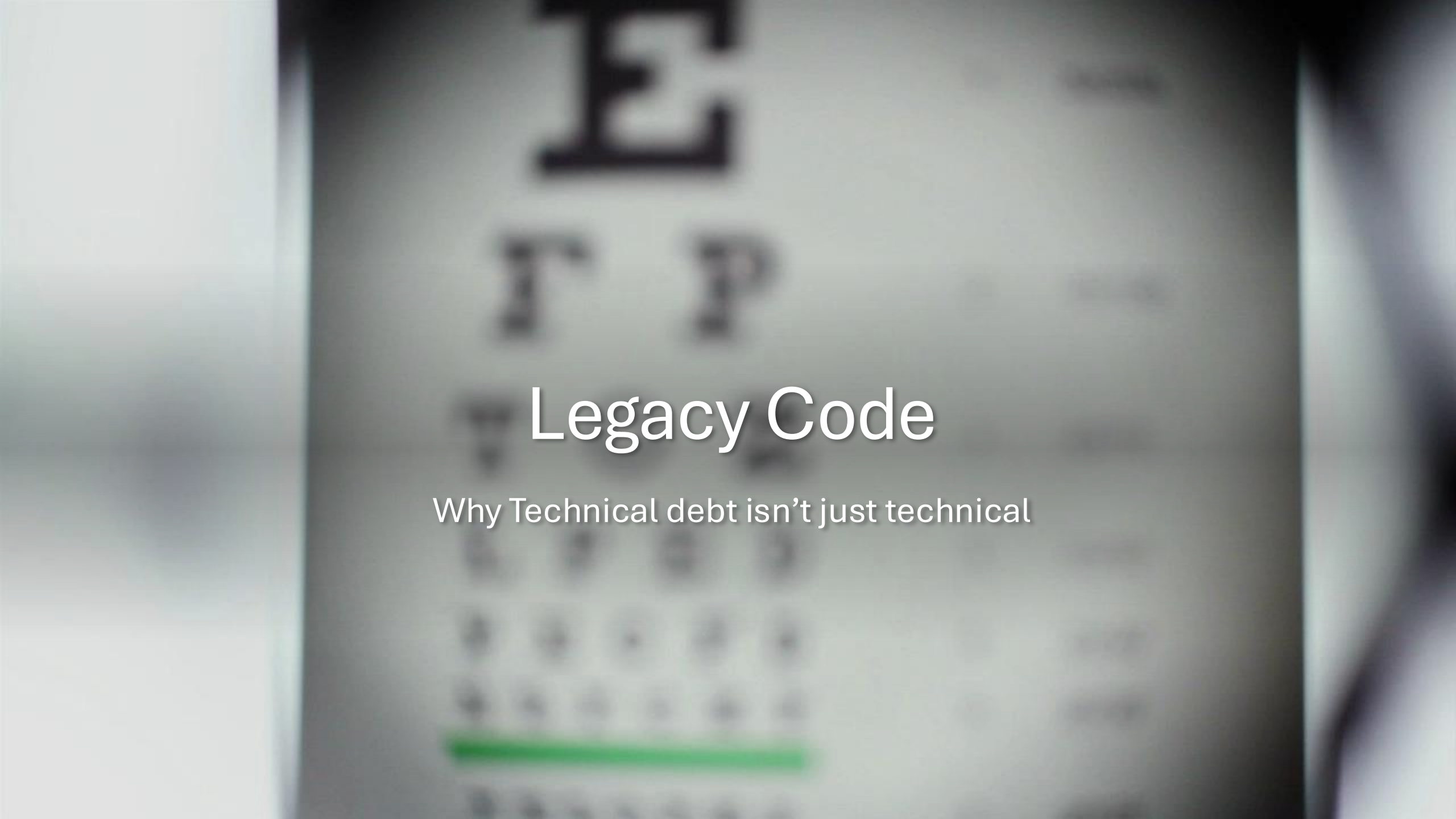
We've identified a problematic file but it's still huge!

Let's *X-Ray* it to find the most problematic functions.

Function	Change Frequency	Lines of Code	Cyclomatic Complexity	Overloaded Functions?	
attach (top-level context)	103	109	9	1	 
attach.recordMount	37	48	8	1	 
attach.handleCommitFiberRoot	32	89	19	1	 
attach.flushPendingEvents	30	105	15	1	 
attach.inspectElement	30	93	12	1	 
attach.flushInitialOperations	29	51	6	1	 
attach.updateFiberRecursively	24	220	48	1	 
attach.inspectElementRaw	24	180	45	1	 
attach.recordUnmount	18	50	9	1	 

1



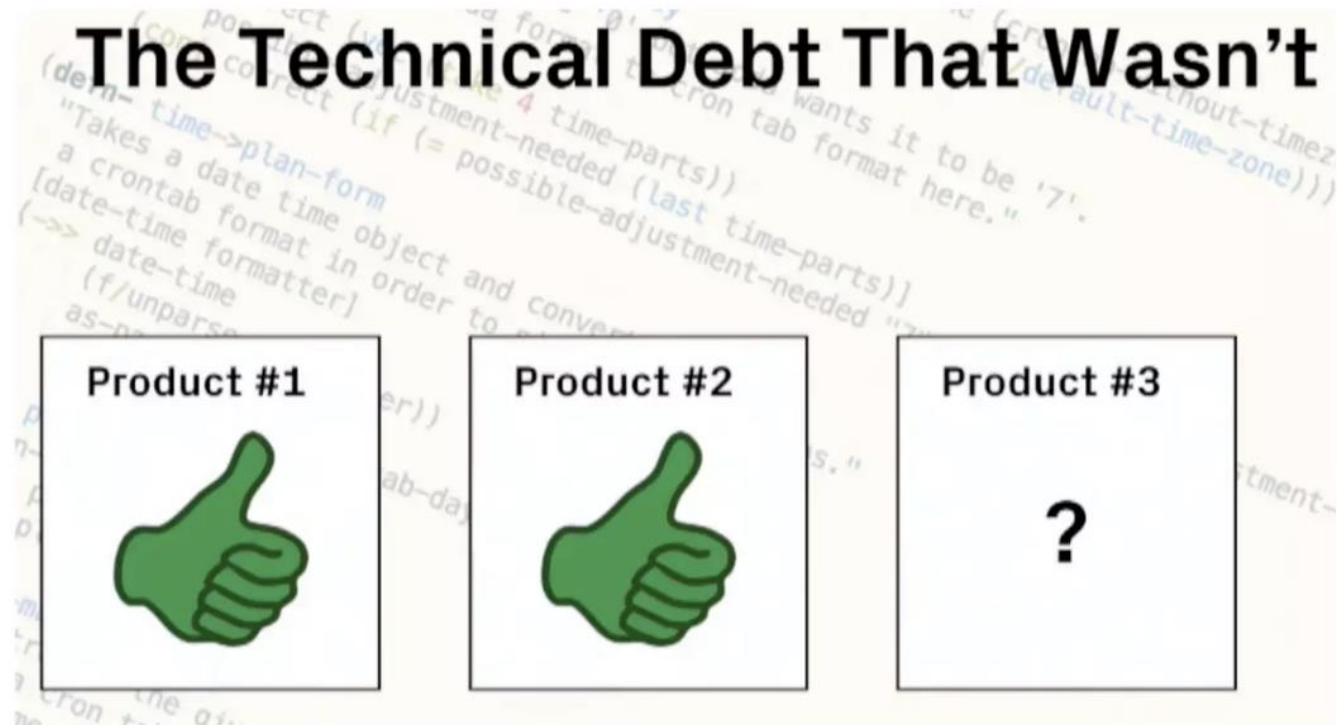


Legacy Code

Why Technical debt isn't just technical

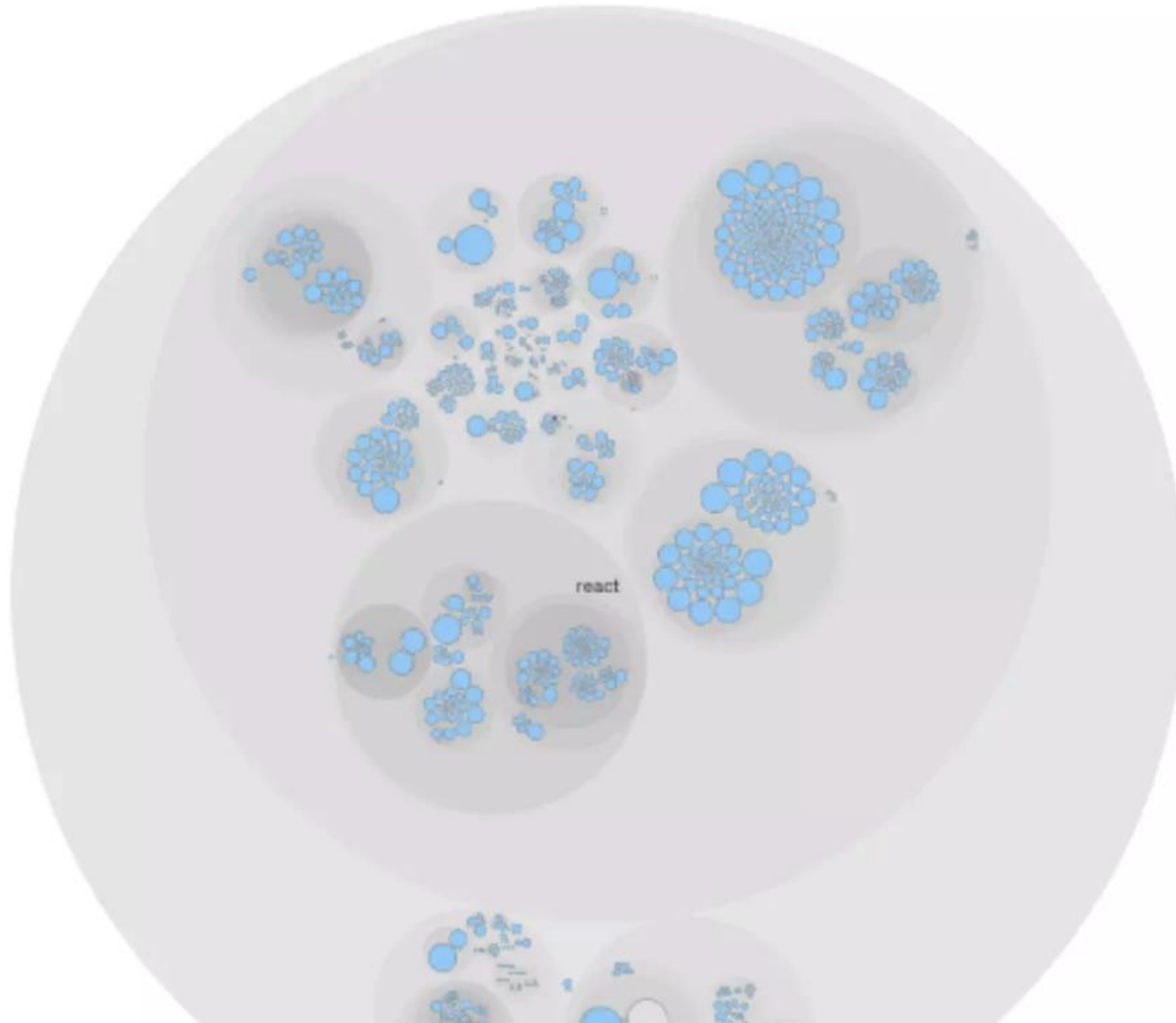
LEGACY CODE

- Legacy code is typically used to describe the code that:
 - lacks in quality (relative perspective)
 - we didn't write ourselves



HOW QUICKLY CAN YOU TURN YOUR CODEBASE INTO LEGACY CODE?

Simulate the effects of a planned off-boarding if some developers leave your organization 



Legend

-  Knowledge
-  Current Loss
-  Simulated Loss
-  Off-Boarding Risk
-  Inconclusive

/ 

Directory: react

 react

Developers

Select one or more developers to simulate their departure from your organization.

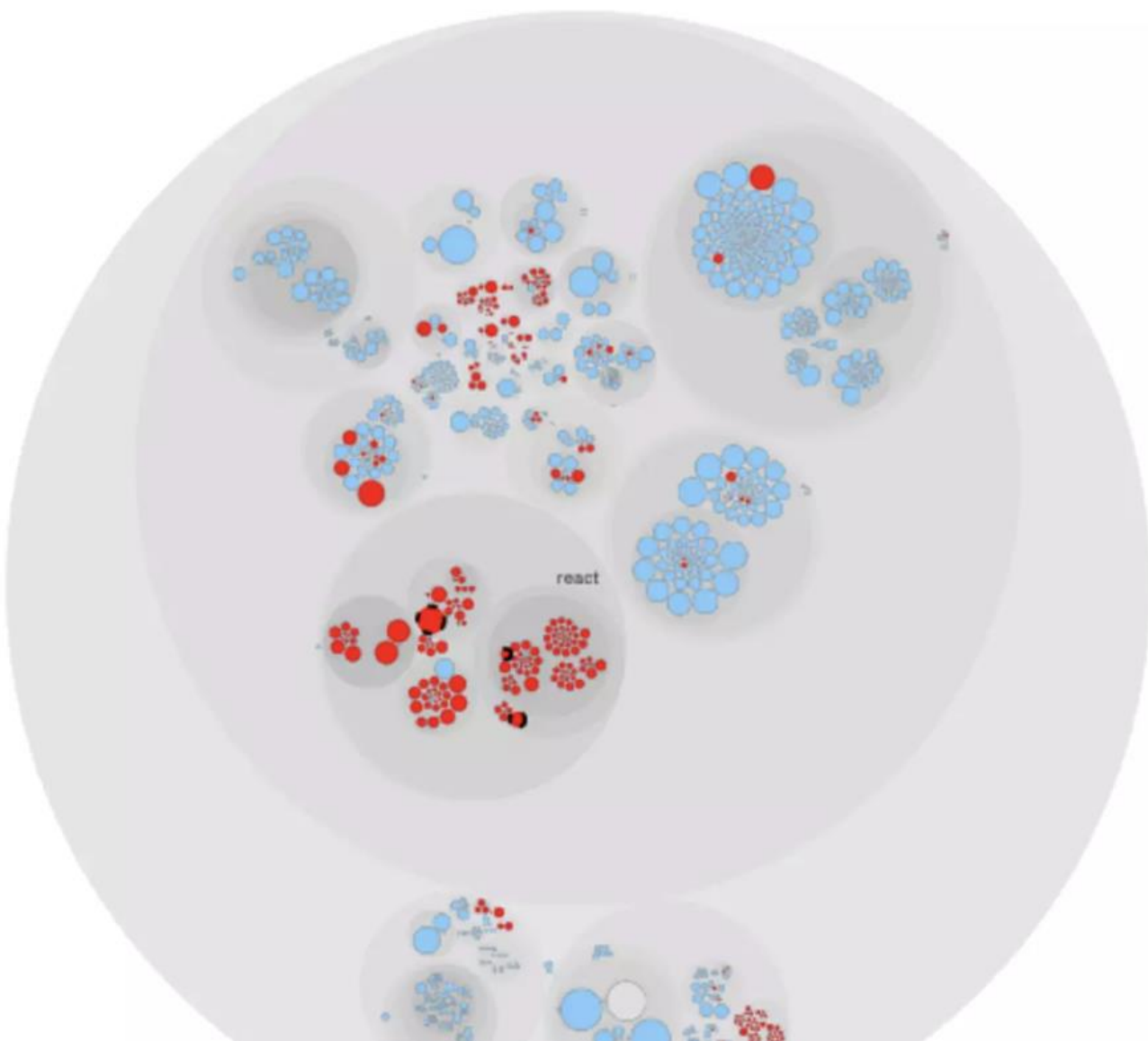
Filter developer names:

brian 

☐ Brian Vaughn

AFTER THEY LEAVE . . .

Simulate the effects of a planned off-boarding if some developers leave your organization 



Legend

-  Knowledge
-  Current Loss
-  Simulated Loss
-  Off-Boarding Risk
-  Inconclusive

/ 

Directory: react

 react

Simulated offboarded authors

 ☒ Brian Vaughn

Developers

Select one or more developers to simulate their departure from your organization.

Filter developer names:

brian



 ☒ Brian Vaughn

THERE'S MUCH MORE IN CODESCENE

- Change coupling
- Microservices
 - Shotgun surgery
 - Team conflicts
 - Technical sprawl
- Proactive warnings
- Retrospectives
- Delivery Performance
- Branch Analyses

TO CONCLUDE...

- Technical debt is a real problem regardless of programming language
- There's a huge amount of useful information stored in your version control system
- Ultimately, you need to rely on human expertise
- Support your developer's judgment and experience with data to get the highest ROI

Resources

- <https://www.adamtornhill.com>
- [Tools: https://www.adamtornhill.com/code/crimescenetools.htm](https://www.adamtornhill.com/code/crimescenetools.htm)